

Rapport Technique

Projet de Fin d'Année

Système de Prédiction de Réadmission Hospitalière (30 jours)

Ce document présente l'ensemble du travail réalisé dans le cadre du projet de fin d'année : nettoyage des données, entraînement des modèles de Machine Learning, déploiement via FastAPI et Streamlit, ainsi que la conception UML du système.

Dataset : hospital_readmissions_30k.csv - 30 000 patients

Modèles : Logistic Regression · Random Forest · XGBoost

Outils : Python · scikit-learn · MLflow · FastAPI · Streamlit · SHAP

Auteur : Salma Chah

Date : Avril 2026

1. Contexte et objectif

L'objectif de ce projet est de prédire si un patient sera réadmis à l'hôpital dans les 30 jours qui suivent sa sortie.

Ce problème est une classification binaire supervisée : la cible est `readmitted_30_days` (Oui / Non).

Un tel système permet aux médecins d'identifier en avance les patients à risque élevé afin d'adapter la prise en charge à la sortie.

2. Dataset

Élément	Détail
Fichier	data/hospital_readmissions_30k.csv
Nombre de patients	30 000 lignes
Séparateur	point-virgule (;)
Variable cible	<code>readmitted_30_days</code> (Yes / No)
Type de problème	Classification binaire

Variables disponibles avant nettoyage :

Variable	Type	Description
<code>patient_id</code>	Identifiant	Numéro unique du patient (non prédictif)
<code>age</code>	Numérique	Âge du patient
<code>cholesterol</code>	Numérique	Taux de cholestérol
<code>bmi</code>	Numérique	Indice de masse corporelle
<code>blood_pressure</code>	Texte	Pression artérielle format "130/72"
<code>medication_count</code>	Numérique	Nombre de médicaments prescrits
<code>length_of_stay</code>	Numérique	Durée d'hospitalisation (jours)
<code>diabetes</code>	Catégoriel	Yes / No
<code>hypertension</code>	Catégoriel	Yes / No
<code>gender</code>	Catégoriel	Male / Female / Other
<code>discharge_destination</code>	Catégoriel	Destination à la sortie
<code>readmitted_30_days</code>	Cible	Yes / No

3. Nettoyage et prétraitement des données

Le pipeline complet est implémenté dans `src/preprocess.py`.

Il comporte 7 étapes exécutées dans un ordre précis pour éviter toute fuite de données (data leakage).

Étape 1 - Chargement

```
df = pd.read_csv("data/hospital_readmissions_30k.csv", sep=";")
```

Le fichier utilise un séparateur ;, ce qui est courant dans les exports hospitaliers.

Résultat : 30 000 lignes × 12 colonnes.

Étape 2 - Suppression de l'identifiant patient

```
df.drop(columns=["patient_id"])
```

patient_id est un identifiant unique sans valeur prédictive.

Le conserver créerait du bruit et risquerait de faire mémoriser des patients au modèle.

Étape 3 - Décomposition de blood_pressure

La colonne blood_pressure contient des valeurs au format chaîne de caractères comme "130/72".

Un modèle ML ne peut pas exploiter directement une chaîne de texte.

Solution : on décompose en deux colonnes numériques :

```
bp = df["blood_pressure"].str.split("/", expand=True).astype(float)
df["bp_sys"] = bp[0]    # pression systolique (ex : 130)
df["bp_dia"] = bp[1]    # pression diastolique (ex : 72)
df.drop(columns=["blood_pressure"])
```

Résultat : 1 colonne texte -> 2 colonnes numériques utilisables.

Étape 4 - Encodage des variables binaires (Yes/No -> 0/1)

Les colonnes diabetes, hypertension et readmitted_30_days contiennent "Yes" ou "No".

Les algorithmes ML n'acceptent que des valeurs numériques.

```
df["diabetes"] = df["diabetes"].map({"Yes": 1, "No": 0})
df["hypertension"] = df["hypertension"].map({"Yes": 1, "No": 0})
df["readmitted_30_days"] = df["readmitted_30_days"].map({"Yes": 1, "No": 0})
```

Étape 5 - Encodage One-Hot de gender et discharge_destination

Ces colonnes ont plusieurs modalités (Male, Female, Other / différentes destinations).

On utilise `pd.get_dummies` pour créer une colonne par modalité :

```
df = pd.get_dummies(df, columns=["gender"], prefix="gender", dtype=int)
df = pd.get_dummies(df, columns=["discharge_destination"], prefix="dest", dtype=int)
```

> Pourquoi `drop_first=False` ?

> On conserve toutes les modalités pour une meilleure interprétabilité avec SHAP.

Étape 6 - Split Train / Test (80 / 20, stratifié)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Le paramètre `stratify=y` garantit que le ratio de patients réadmis est identique dans le train et dans le test. Cela évite qu'un ensemble soit biaisé par hasard.

Ensemble	Taille
Train	~24 000 patients
Test	~6 000 patients

Étape 7 - Normalisation (StandardScaler)

Les colonnes numériques continues ont des échelles très différentes (ex : `age` dans `[0, 100]` vs `cholesterol` dans `[100, 400]`).

Un scaler homogénéise les plages de valeurs.

```
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols]) # transform UNIQUEMENT
```

> Règle critique : fit uniquement sur le train.

> Si on fittait sur le test, le modèle "verrait" les données de test avant d'être évalué -> fuite de données.

Colonnes normalisées : `age`, `cholesterol`, `bmi`, `medication_count`, `length_of_stay`, `bp_sys`, `bp_dia`.

Étape 8 - Rééquilibrage SMOTE (sur train uniquement)

Le dataset est déséquilibré : les patients non réadmis sont plus nombreux que les patients réadmis.

Un modèle entraîné sur données déséquilibrées apprend à dire "Non" presque tout le temps.

SMOTE (Synthetic Minority Over-sampling Technique) génère des exemples synthétiques de la classe minoritaire (réadmis = 1) par interpolation entre voisins réels.

```
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)
```

Classe	Avant SMOTE	Après SMOTE
Non réadmis (0)	majoritaire	maintenu
Réadmis (1)	minoritaire	égalisé

> Important : SMOTE est appliqué uniquement sur le train.

> Le test reste dans la distribution réelle pour une évaluation honnête.

Les artefacts produits (scaler.pkl, feature_names.pkl) sont sauvegardés dans models/.

4. Entraînement des modèles ML

Pipeline implémenté dans src/train.py, avec tracking automatique via MLflow.

Trois modèles sont comparés :

Modèle	Stratégie	Détail
Logistic Regression	Baseline (pas de tuning)	C=1.0, max_iter=1000, solver=lbfgs
Random Forest	GridSearchCV (3 folds)	n_estimators, max_depth, min_samples_leaf
XGBoost	GridSearchCV (3 folds)	n_estimators, max_depth, learning_rate, subsample

Métriques calculées sur le test set

Pour chaque modèle : Accuracy, Precision, Recall, F1-Score, AUC-ROC.

La métrique principale de sélection est l'AUC-ROC car le dataset est déséquilibré.

Sélection du meilleur modèle

Le modèle avec le meilleur AUC-ROC est automatiquement :

- sauvegardé dans models/model.pkl
- tagué best_model = true dans MLflow
- utilisé par l'API FastAPI

Artefacts générés

- models/metrics.json - métriques de tous les modèles

- models/feature_importance.csv - importance des features
- models/artifacts/confusion_matrix_*.png - matrices de confusion
- models/artifacts/roc_curves_comparison.png - courbes ROC comparées

5. Déploiement

API FastAPI (api/main.py)

L'API expose le modèle entraîné sous forme de service REST.

Endpoint	Méthode	Description
/	GET	Statut de l'API
/health	GET	Vérifie si le modèle est chargé
/predict	POST	Reçoit les données patient, retourne probabilité + niveau de risque
/model/info	GET	Informations sur le modèle chargé

Niveaux de risque calculés automatiquement :

Probabilité	Niveau
< 30 %	Low
30 % - 60 %	Medium
> 60 %	High

Dashboard Streamlit (dashboard/app.py)

Interface graphique interactive avec 4 pages :

Page	Contenu
Home	Résumé du dataset (nombre de patients, taux de réadmission)
Data Exploration	Distribution des variables, valeurs manquantes, visualisations
Prediction	Formulaire de saisie patient -> appel API -> jauge de risque
Model Performance	Métriques, top 20 feature importances

6. Conception - Diagrammes UML

Les diagrammes sont générés par uml/use_case.py et uml/conception.py.

Diagramme de cas d'utilisation (uml/use_case.png)

Deux acteurs identifiés :

Médecin (utilisateur principal)

1. Saisir les données du patient (age, BMI, diabète, tension...)
2. Obtenir une prédiction de réadmission (probabilité en %)
3. Voir le niveau de risque (Low / Medium / High)
4. Voir l'explication SHAP (pourquoi cette prédiction ?)
5. Consulter l'historique des prédictions

Admin (exploitation et supervision)

1. Consulter les statistiques du dataset (30 000 patients)
2. Comparer les 3 modèles ML (XGBoost, RandomForest, LogisticRegression)
3. Voir les métriques MLflow (AUC-ROC, F1-Score, Recall)
4. Gérer les données (importer un nouveau dataset)

Relations UML :

- Obtenir prédiction <<include>> Saisir données - une prédiction nécessite obligatoirement des données
- Voir explication SHAP <<include>> Obtenir prédiction - l'explication dépend d'une prédiction existante
- Voir niveau de risque <<extend>> Obtenir prédiction - le niveau de risque est une extension optionnelle

Diagramme de séquence (uml/sequence.png)

Flux complet d'une prédiction :

```

Médecin -> Dashboard Streamlit : saisie du formulaire patient
Dashboard -> API FastAPI : POST /predict {données JSON}
API : prétraitement + encoding features
API -> Modèle XGBoost : predict_proba(X_scaled)
XGBoost -> API : probabilité brute (0..1)
API -> SHAP Explainer : shap_values(X)
SHAP -> API : valeurs SHAP + features importantes
API : calcul niveau de risque (Low/Medium/High)
API -> Dashboard : probabilité + niveau risque + explication SHAP
Dashboard -> Médecin : affichage jauge + niveau + graphique SHAP
  
```

7. Stack technique

Catégorie	Technologie
Langage	Python 3
ML	scikit-learn, XGBoost, imbalanced-learn (SMOTE)

Tracking

MLflow

Explicabilité

SHAP

API

FastAPI + Uvicorn

Dashboard

Streamlit + Plotly

Visualisation

Matplotlib, Seaborn

Données

Pandas, NumPy

8. Architecture du projet

```
PFA_readmissionhospi/
|-- data/
|   |-- hospital_readmissions_30k.csv    # Dataset brut
|-- src/
|   |-- preprocess.py                    # Pipeline nettoyage + SMOTE
|   |-- train.py                         # Entraînement 3 modèles + MLflow
|-- api/
|   |-- main.py                          # API FastAPI /predict
|-- dashboard/
|   |-- app.py                           # Interface Streamlit
|-- models/
|   |-- model.pkl                        # Meilleur modèle sérialisé
|   |-- scaler.pkl                       # StandardScaler fitté
|   |-- feature_names.pkl                # Ordre des features
|   |-- metrics.json                     # Métriques de comparaison
|-- uml/
|   |-- use_case.py                      # Génère use_case.png
|   |-- use_case.png                     # Diagramme cas d'utilisation
|   |-- conception.py                    # Génère usecase.png + sequence.png
|   |-- sequence.png                     # Diagramme de séquence
|-- mlruns/                              # Runs MLflow
`-- notebooks/
    |-- 01_eda.ipynb                     # Analyse exploratoire
    |-- 02_preprocessing.ipynb           # Prototypage prétraitement
```

Rapport généré le 15 avril 2026 - Projet de Fin d'Année (PFA)